

DOCKER & DEVOPS

Support de cours — Conteneurisation, intégration continue
et bonnes pratiques d'automatisation

Niveau : Débutant / Intermédiaire

Prérequis : bases de la ligne de commande Linux, notions de développement logiciel

8 pages — Document pédagogique

1. Introduction au DevOps

Le **DevOps** (contraction de Development et Operations) est une culture et un ensemble de pratiques visant à rapprocher les équipes de développement et d'exploitation, afin de livrer des logiciels plus rapidement, plus fréquemment et de manière plus fiable. Il ne s'agit pas d'un outil, mais d'une philosophie de travail soutenue par l'automatisation.

Définition clé

Le DevOps vise à unifier le cycle de vie du logiciel — développement, tests, livraison, exploitation — grâce à l'automatisation, la collaboration et le monitoring continu.

Les piliers du DevOps

- **Culture** — collaboration entre équipes, responsabilité partagée
- **Automatisation** — réduire les tâches manuelles et répétitives
- **Mesure** — surveiller les performances et la qualité en continu
- **Partage** — documentation et retours d'expérience communs

Le cycle DevOps (boucle infinie)

Planifier → Développer → Construire (Build) → Tester → Livrer (Release) → Déployer → Exploiter → Surveiller (Monitor), puis retour à la planification. Ce cycle continu permet des livraisons fréquentes et une amélioration constante.

Pourquoi Docker s'inscrit dans le DevOps ?

Docker facilite l'automatisation des environnements : il garantit qu'une application fonctionne de manière identique du poste du développeur jusqu'à la production, ce qui est au cœur des objectifs du DevOps.

2. Les concepts fondamentaux de Docker

Docker est une plateforme de conteneurisation qui permet d'empaqueter une application avec toutes ses dépendances dans une unité portable et isolée : le **conteneur**.

Conteneur vs machine virtuelle

Contrairement à une machine virtuelle qui embarque un système d'exploitation complet, un conteneur partage le noyau du système hôte. Il est donc beaucoup plus léger, démarre en quelques secondes et consomme moins de ressources.

Critère	Machine virtuelle	Conteneur
Poids	Plusieurs Go	Quelques Mo
Démarrage	Minutes	Secondes
Isolation	Totale (hyperviseur)	Processus isolés
OS	Un par VM	Noyau partagé

Vocabulaire essentiel

- **Image** — modèle en lecture seule décrivant un environnement (OS, dépendances, code)
- **Conteneur** — instance en cours d'exécution d'une image
- **Dockerfile** — fichier texte décrivant les étapes de construction d'une image
- **Registry** — dépôt d'images (ex. Docker Hub) permettant de les partager
- **Volume** — mécanisme de persistance des données en dehors du conteneur

3. Créer une image avec un Dockerfile

Un **Dockerfile** est un ensemble d'instructions permettant de construire une image de façon reproductible. Chaque instruction crée une nouvelle couche (layer) dans l'image.

Exemple : application Python

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["python", "app.py"]
```

Instructions clés

- **FROM** — image de base à partir de laquelle construire
- **WORKDIR** — définit le répertoire de travail dans le conteneur
- **COPY** — copie des fichiers depuis l'hôte vers l'image
- **RUN** — exécute une commande lors de la construction de l'image
- **EXPOSE** — documente le port utilisé par l'application
- **CMD** — commande exécutée au démarrage du conteneur

Construire et exécuter

```
docker build -t mon-app:1.0 .
docker run -p 8000:8000 mon-app:1.0
```

Bonne pratique

Utiliser des images de base légères (slim, alpine) et regrouper les instructions RUN pour limiter le nombre de couches et la taille finale de l'image.

4. Les commandes Docker essentielles

La CLI Docker permet de gérer l'ensemble du cycle de vie des images et des conteneurs depuis le terminal.

Gestion des images

```
docker images          # lister les images locales
docker pull nginx:latest # telecharger une image
docker rmi mon-app:1.0  # supprimer une image
```

Gestion des conteneurs

```
docker ps              # conteneurs en cours d'execution
docker ps -a           # tous les conteneurs
docker run -d --name web nginx # lancer en arriere-plan
docker stop web        # arreter un conteneur
docker rm web          # supprimer un conteneur
docker logs -f web     # suivre les logs en direct
docker exec -it web bash # ouvrir un shell dans le conteneur
```

Volumes et réseaux

```
docker volume create data_db
docker run -v data_db:/var/lib/mysql mysql

docker network create mon-reseau
docker run --network mon-reseau nginx
```

Nettoyage

```
docker system prune -a # supprime images, conteneurs et caches inutilises
```

5. Orchestrer plusieurs services avec Docker Compose

Docker Compose permet de définir et de lancer des applications composées de plusieurs conteneurs (application, base de données, cache...) à l'aide d'un unique fichier de configuration YAML.

Exemple : application web + base de données

```
version: "3.9"

services:
  web:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/app

  db:
    image: postgres:16
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=app
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Commandes principales

```
docker compose up -d      # demarrer tous les services
docker compose ps         # etat des services
docker compose logs -f    # suivre les logs
docker compose down       # arreter et supprimer les services
```

À retenir

Docker Compose est idéal pour le développement local et les environnements de test ; pour la production à grande échelle, on utilise plutôt un orchestrateur comme Kubernetes.

6. Intégration et déploiement continu (CI/CD)

L'**intégration continue** (CI) consiste à fusionner et tester automatiquement le code à chaque modification. Le **déploiement continu** (CD) automatise ensuite la mise en production des versions validées.

Pipeline CI/CD typique

- **1. Commit / Push** — le développeur pousse son code vers le dépôt Git
- **2. Build** — compilation et construction de l'image Docker
- **3. Tests automatisés** — tests unitaires, d'intégration, analyse de qualité
- **4. Publication** — envoi de l'image vers un registry (Docker Hub, GitLab Registry)
- **5. Déploiement** — mise à jour automatique de l'environnement cible
- **6. Monitoring** — surveillance post-déploiement, alertes en cas d'anomalie

Exemple : pipeline GitHub Actions

```
name: CI/CD

on: [push]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Construire l'image Docker
        run: docker build -t mon-app:$GITHUB_SHA .
      - name: Lancer les tests
        run: docker run mon-app:$GITHUB_SHA pytest
      - name: Publier l'image
        run: docker push mon-app:$GITHUB_SHA
```

Outils courants

- GitHub Actions, GitLab CI/CD, Jenkins
- Docker Hub, GitLab Container Registry, Amazon ECR
- Terraform et Ansible pour l'infrastructure as code

7. Orchestration et supervision

Pourquoi Kubernetes ?

Lorsqu'une application est déployée à grande échelle sur plusieurs serveurs, **Kubernetes** (K8s) permet d'automatiser le déploiement, la mise à l'échelle et la gestion des conteneurs : redémarrage automatique en cas de panne, répartition de charge, mises à jour progressives (rolling updates).

Concepts clés de Kubernetes

- **Pod** — plus petite unité déployable, contenant un ou plusieurs conteneurs
- **Deployment** — gère le cycle de vie et le nombre de réplicas d'un pod
- **Service** — expose un ensemble de pods via une adresse stable
- **Cluster** — ensemble de machines (nœuds) exécutant les conteneurs

Infrastructure as Code (IaC)

L'infrastructure est décrite sous forme de fichiers de configuration versionnés (plutôt que configurée manuellement), ce qui la rend reproductible, traçable et automatisable. Outils courants : **Terraform**, **Ansible**, **Pulumi**.

Monitoring et observabilité

Une démarche DevOps mature repose sur une supervision continue des applications et de l'infrastructure, à travers trois piliers : les **métriques** (Prometheus, Grafana), les **logs** centralisés (ELK Stack, Loki) et le **traçage** distribué (Jaeger, OpenTelemetry).

À retenir

Docker répond au besoin d'empaqueter une application ; Kubernetes répond au besoin de l'orchestrer à grande échelle. Les deux sont complémentaires.

8. Bonnes pratiques et conclusion

Bonnes pratiques Docker

- Utiliser des images officielles et légères (alpine, slim)
- Ne jamais stocker de secrets en clair dans une image
- Utiliser un fichier .dockerignore pour exclure les fichiers inutiles
- Construire des images multi-stage pour réduire leur taille
- Attribuer des tags précis aux images (éviter uniquement 'latest')
- Exécuter les conteneurs avec un utilisateur non-root

Exemple de build multi-stage

```
# Etape 1 : build
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install && npm run build

# Etape 2 : image finale allégée
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

Points clés à retenir

- Le DevOps est une culture d'automatisation et de collaboration continue
- Docker isole et rend portable une application grâce aux conteneurs
- Le Dockerfile décrit la construction reproductible d'une image
- Docker Compose orchestre plusieurs services en local
- La CI/CD automatise tests, build et déploiement à chaque changement
- Kubernetes prend le relais pour l'orchestration à grande échelle

Pour aller plus loin

- Documentation officielle Docker (docs.docker.com)
- Kubernetes : concepts avancés (Helm, Ingress, autoscaling)
- Sécurité des conteneurs (scan d'images, gestion des secrets)
- Monitoring avancé avec Prometheus et Grafana

Conclusion

Docker et le DevOps transforment la manière de concevoir, tester et livrer des logiciels : automatiser, standardiser et surveiller en continu permettent de livrer plus vite, avec plus de fiabilité. La pratique régulière sur des projets réels reste la meilleure voie de progression.